Cairo University

Faculty of Engineering

Electronics and Electrical Communications Engineering

Assignment 2

ELC 4028 – Artificial Neural Networks

ID	Section	Name
9220399	2	Shahd Gamal Mahmoud Hussein
9220411	2	Salah El-Din Hassan Hassan Mohamed
9220984	4	Yousef Khaled Omar Mahmoud

Supervised by:

Dr. Mohsen Rashwan
Dr. Mohamed Motaz

Contents

Problem 1 **3**

1 Feature Extraction and MLP Benchmark **3**
1.1 Dataset and Preprocessing . . . 3
1.2 Feature Construction . . . 3
1.2.1 Discrete Cosine Transform Features . . . 3
1.2.2 Principal Component Analysis Features . . . 3
1.2.3 Autoencoder Features . . . 3
1.3 Classifier Design . . . 3
1.4 Integration with the Assignment 1 Benchmark . . . 4
1.5 Summary . . . 4

Problem 2 **5**

2 CNN Experiments (LeNet-style) **5**
2.1 Design Decisions . . . 5
2.2 Parameters of the Experiment . . . 6
2.3 Results and Findings . . . 6
2.4 Discussion . . . 7
2.5 Conclusion and Recommendations . . . 7

Problem 3 **8**

3 Placeholder **8**
3.1 Results . . . 8

Problem 4 **9**

4 Placeholder **9**
4.1 Results . . . 9

Problem 5 **10**

5 Placeholder **10**
5.1 Results . . . 10

Problem 6 **11**

6 Placeholder **11**
6.1 Results . . . 11

Conclusion **12**

List of Figures

1 LeNet-5 architecture (baseline). 5

Problem 1

1 Feature Extraction and MLP Benchmark

1.1 Dataset and Preprocessing

The Reduced MNIST dataset was downloaded through `kagglehub` and already provided in labeled training and testing folders, one subdirectory per digit class. In our implementation, each image was loaded from disk, converted again to grayscale to ensure a single-channel input, and normalized to the range $[0, 1]$ by dividing pixel intensities by 255.

The label of each sample was taken directly from the folder name. No relabeling or manual reshuffling was introduced, which keeps the preprocessing aligned with the assignment dataset structure.

1.2 Feature Construction

The assignment required three feature representations: DCT, PCA, and autoencoder features. We implemented each feature extractor separately so that the same classifier could be evaluated on all representations under a fair comparison.

1.2.1 Discrete Cosine Transform Features

For the DCT representation, each grayscale image was transformed using a two-dimensional discrete cosine transform with `scipy.fft.dctn`. We used the orthonormal DCT-II variant and retained the top-left 15×15 low-frequency block, which captures the most informative spatial structure while reducing the original 28×28 image to a compact 225-dimensional vector.

1.2.2 Principal Component Analysis Features

For PCA, each image was first flattened into a 784-dimensional vector and then projected onto the principal subspace using `sklearn.decomposition.PCA`. We configured PCA with `n_components=0.95`, which preserves 95% of the variance in the training data.

1.2.3 Autoencoder Features

We implemented it as part of the same feature pipeline so that its learned latent space could be compared directly with DCT and PCA.

The autoencoder is implemented as a multilayer perceptron regressor with symmetric hidden layers `(128, 64, 32, 64, 128)`. It is trained to reconstruct the input image using ReLU activations and early stopping. The bottleneck is the middle hidden layer with 32 units. After training, we pass the data forward layer by layer and extract the bottleneck activations as the learned feature vector for each image.

1.3 Classifier Design

After extracting features, we trained a multilayer perceptron classifier on each feature set. The classifier uses ReLU activations, Adam optimization, early stopping, and a maximum of 120 iterations.

We evaluated three hidden-layer configurations:

- one hidden layer: `(512,)`
- three hidden layers: `(512, 256, 128)`
- four hidden layers: `(512, 256, 128, 64)`

These configurations were chosen to study the effect of model depth on classification performance while keeping the feature extractor fixed.

1.4 Integration with the Assignment 1 Benchmark

The autoencoder implementation was also reused later in Problem 2 because the assignment table required us to rerun the Assignment 1 Part 2 benchmark using this feature.

1.5 Summary

In summary, Problem 1 focuses on the feature-extraction stage of the experiment. The three required features were implemented exactly as separate pipelines, and the same MLP classifier was applied to all of them using three hidden-layer configurations. The detailed accuracy and timing results are documented in the results table of Problem 2.

Problem 2

2 CNN Experiments (LeNet-style)

2.1 Design Decisions

We implemented a baseline `LeNet5` with the following structure:

- Input image: $28 \times 28 \times 1$
- Convolutional layer 1: 5×5, 6 filters $\rightarrow 24 \times 24 \times 6$
- Average pooling $2 \times 2 \rightarrow 12 \times 12 \times 6$
- Convolutional layer 2: 5×5, 16 filters $\rightarrow 8 \times 8 \times 16$
- Average pooling $2 \times 2 \rightarrow 4 \times 4 \times 16$
- Flatten $\rightarrow$ 256 features
- Fully connected layers: $256 \rightarrow 120 \rightarrow 84 \rightarrow 10$ logits

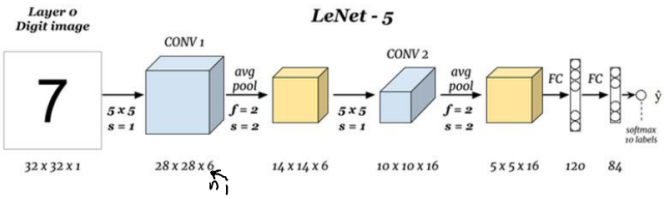

Figure 1: LeNet-5 architecture (baseline).

Variants added to investigate capacity and regularisation effects:

Wide: The spatial flow becomes:

- Input image: $28 \times 28 \times 1$
- Convolutional layer 1: 5×5, **16** filters $\rightarrow 24 \times 24 \times$ **16**
- Average pooling $2 \times 2 \rightarrow 12 \times 12 \times$ **16**
- Convolutional layer 2: 5×5, **32** filters $\rightarrow 8 \times 8 \times$ **32**
- Average pooling $2 \times 2 \rightarrow 4 \times 4 \times$ **32**
- Flatten $\rightarrow$ **512** features
- Fully connected layers: **512** $\rightarrow 120 \rightarrow 84 \rightarrow 10$ logits

Deep: The spatial flow becomes:

- Input image: $28 \times 28 \times 1$
- Convolutional layer 1: 5×5, 6 filters $\rightarrow 24 \times 24 \times 6$
- Average pooling $2 \times 2 \rightarrow 12 \times 12 \times 6$
- Convolutional layer 2: 5×5, 16 filters $\rightarrow 8 \times 8 \times 16$
- Convolutional layer 3: 3×3, **32** filters $\rightarrow 8 \times 8 \times$ **32**

- Average pooling $2 \times 2 \rightarrow 4 \times 4 \times \mathbf{32}$
- Flatten $\rightarrow$ **512** features
- Fully connected layers: **512** $\rightarrow$ 120 $\rightarrow$ 84 $\rightarrow$ 10 logits

Sigmoid: Same as the baseline in terms of spatial sizes and feature dimensions, but with Sigmoid activations instead of ReLU.

Each variant is constructed by a factory function and run with the same training pipeline to ensure fair comparison.

2.2 Parameters of the Experiment

- Learning rate: 0.001.
- Batch size: 128.
- Epochs: 5.
- Metrics: train accuracy, test accuracy, training wall time, testing wall time.

2.3 Results and Findings

The numerical outputs for the baseline and the three variants will be recorded in the consolidated tables below. Cells are left blank as placeholders to be filled with measured accuracies and times.

	DCT		PCA		AutoEncoder	
Classifier	Accuracy	Processing Time	Accuracy	Processing Time	Accuracy	Processing Time
K-means (k=1)	0.9155	0.22174417	0.9165	0.11880670		
K-means (k=4)	0.9590	0.65419497	0.9595	0.62659590		
K-means (k=16)	0.9760	1.94901458	0.9775	1.66873733		
K-means (k=32)	0.9790	3.46641002	0.9775	2.53172698		
SVM (Linear)	0.9815	0.71919631	0.9785	0.59357043		
SVM (Nonlinear)	0.9855	1.84773630	0.9895	2.05351391		

Table 1: Feature-based classifier results (placeholders).

	DCT		PCA		AutoEncoder	
Variations	Accuracy	Processing Time	Accuracy	Processing Time	Accuracy	Processing Time
MLP (1-hidden)						
MLP (3-hidden)						
MLP (4-hidden)						

Table 2: MLP benchmark results (placeholders).

Variations	Accuracy	Training time	Testing time
Baseline			
Wide			
Deep			
Sigmoid			

Table 3: CNN experiments summary (placeholders).

2.4 Discussion

Discuss why the wide or deep variants behaved differently from baseline, how Sigmoid activation affected training dynamics, and any regularisation or optimisation observations (e.g., learning rate sensitivity).

2.5 Conclusion and Recommendations

Summarise best-performing variant, trade-offs between accuracy and compute (training time), and recommendations for further experiments (longer training, data augmentation, normalization).

Problem 3

3 Placeholder

3.1 Results

Place figures and tables for Problem 3 in this file.

Problem 4

4 Placeholder

4.1 Results

Place figures and tables for Problem 4 in this file.

Problem 5

5 Placeholder

5.1 Results

Place figures and tables for Problem 5 in this file.

Problem 6

6 Placeholder

6.1 Results

Place figures and tables for Problem 6 in this file.

Conclusion

Summarize key findings across all problems, reflect on the learning outcomes, and suggest potential future directions or improvements for the experiments conducted.